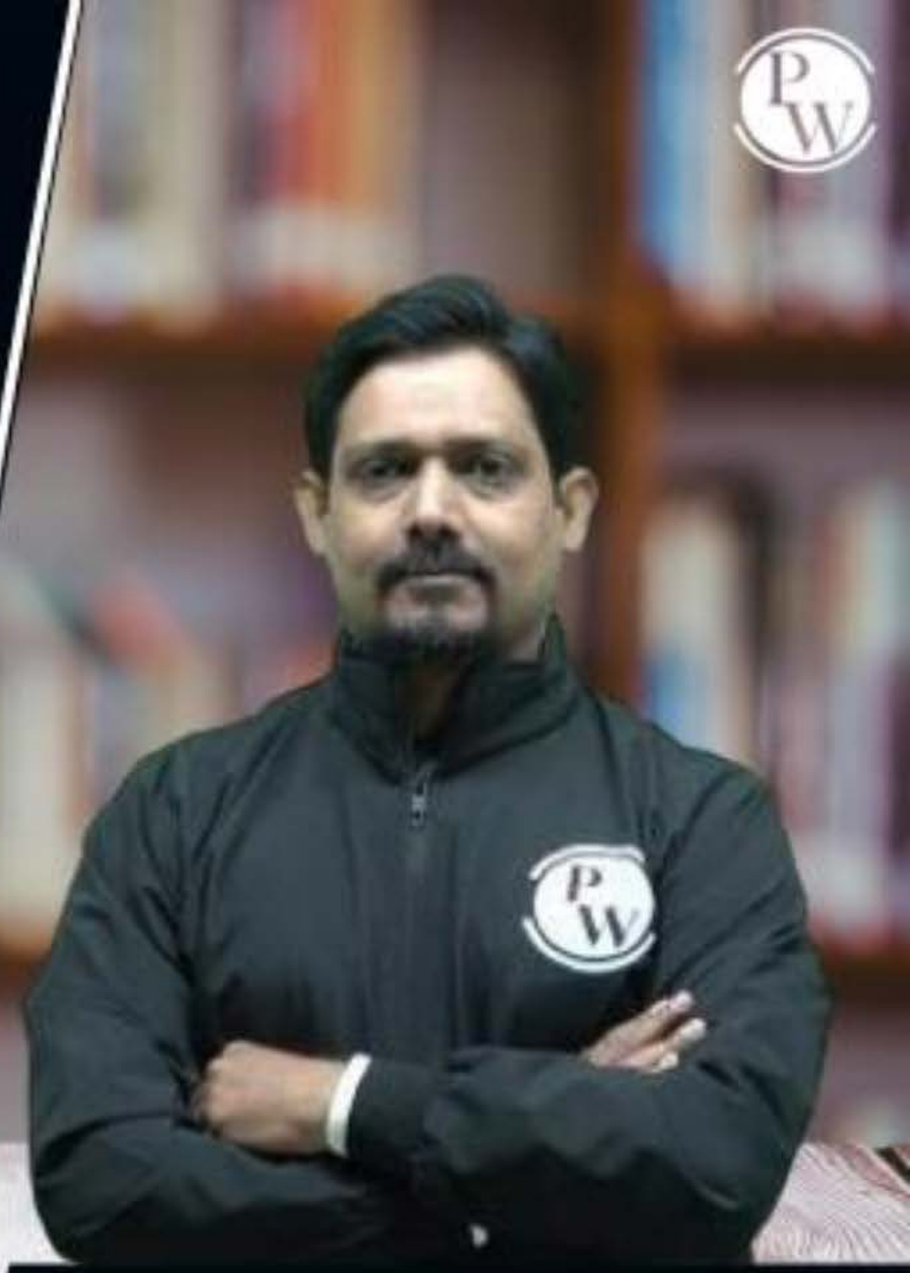
Computer Science & IT

Data Structure & programming

Linked List

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture

Topic	Dynamic allocation Memory
Topic	Malloc calloc free
Topic	Memory leak.
Topic	getnode(), display, Build123
Topic	

Slide

Topics to be Covered

- Topic — NULL pointer Dereferencing
- Topic — Basic Linked list operation
- Topic — Two pointer traversal
- Topic
- Topic

Slide

Topic Question

Consider the function defined below.

```
struct item {
        int data;
        struct item * next;
};
int f(struct item *p) {
    return ((p == NULL) || (p->next == NULL)||
        ((p->data <= p ->next -> data) &&
        f(p->next)));
}
```

For a given linked list p , the function f returns 1 if and only if

1 → 2

False A the list is empty or has exactly one element

B the elements in the list are sorted in non-decreasing order of data value

1 → 1 → 2

C the elements in the list are sorted in non-increasing order of data value ✗

D . not all elements in the list have the same data value

1 → 1 → 1

return 1

Slide

Topic Question

Consider the function defined below.

```
struct item {
        int data;
        struct item * next;
};
int f(struct item *p) {
    return ((p == NULL) || (p->next == NULL)||
        ((p->data <= p ->next -> data) &&
        f(p->next)));
}
```

Single element : 1 → Sorted

Non decreasing

Empty Sorted

Topic Question

If the list contain the value 1→2→3 then number of integer printed will be

```
void foo (Node *ptr){
if (ptr){
        fun (ptr -> link);
        printf ("%d", ptr ->data);
        fun (ptr->link);
        return;
}
```

No. of values printed by list with n element

$$T(n) = 2T(n-1) + 1$$

1→2→3 2→3

$$T(1) = 2T(0) + 1 = 1$$

$$T(2) = 2T(1) + 1 = 2 \times 1 + 1 = 3$$

$$T(3) = 2T(2) + 1 = 2 \times 3 + 1 = 7$$

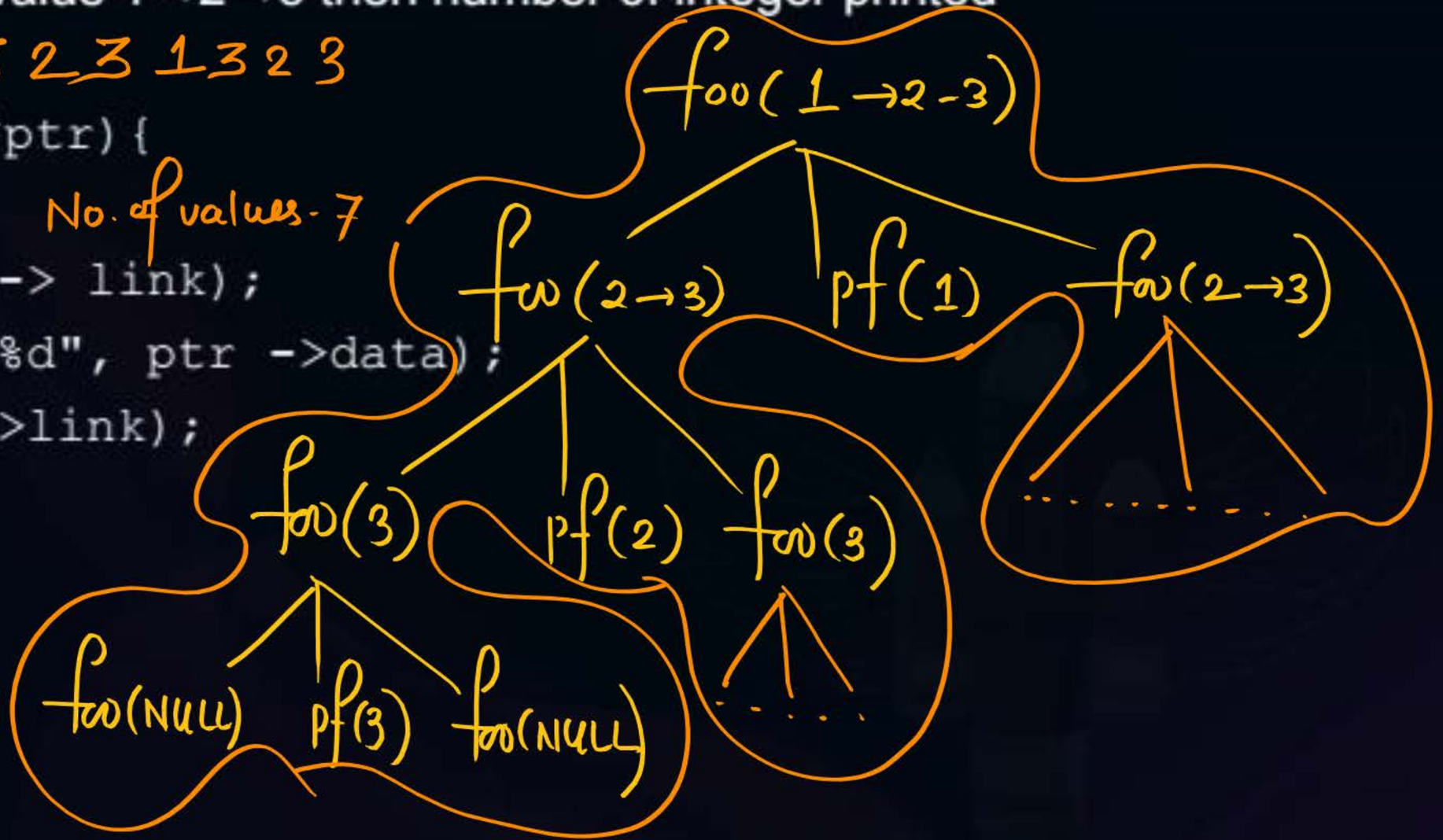
Slide

Topic Question

If the list contain the value 1→2→3 then number of integer printed will be 3 2 3 1 3 2 3

```
void foo (Node *ptr){
if (ptr){
        fun (ptr -> link);
        printf ("%d", ptr ->data);
        fun (ptr->link);
        return;
}
```

No. of values - 7



Consider the function

```
void foo(node *s, int x){
        node *temp,*p =getnode(x);
        if (s==NULL)
                hp = p;
        else{
                while(s->next!=NULL)
                        s = s->next;
                s->next = p;
}
int main(){
        for(int i = 11; i<=81; i+=14)
                foo(hp, i);
        return 0;
}
```

Assuming HP is global variable and holding the value NULL . What will be the content of the list

A . 11 → 25 → 39 → 53 → 67 → 81 → NULL

B. 81 → 67 → 53 → 39 → 25 → 11 → NULL

C. 11 → 14 → 28 → 42 → 56 → 70 → 84 → NULL

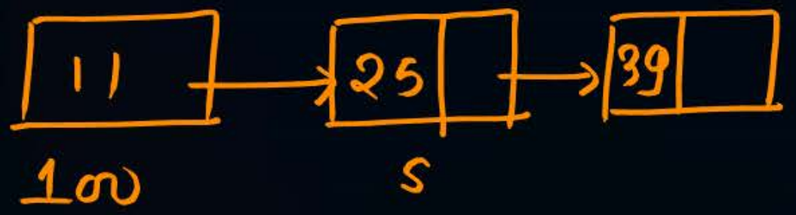
D . 25 → 39 → 53 → 67 → 81 → NULL

Slide

Consider the function

```
void foo(node *s, int x){
        node *temp,*p =getnode(x);
        if (s==NULL)
                hp = p;
        else{
                while(s->next!=NULL)
                        s = s->next;
                s->next = p;
}
int main(){
        for(int i = 11; i<=81; i+=14)
                foo(hp, i);
        return 0;
}
```

h = 100



i = 11
i = 25
i = 39
i = 53
i = 67
i = 81

Slide

Adding element at the end of list

```
Consider the function
void foo(node *s, int x){
        node *temp,*p =getnode(x);
        if (s==NULL)
                hp = p;
        else{
                while(s->next!=NULL)
                        s = s->next;
                s->next = p;
}
int main(){
        for(int i = 11; i<=81; i+=14)
                foo(hp, i);
        return 0;
}
```

← function depends upon length of list.

O(n) worst

Ω(n) Best

hence Θ(n)

```
Consider the function
node *hp;
void bar(node *s, int x){
    node *p =getnode(x);

        p->next = s;
        hp = p;
}
int main(){
    for(int i = 34; i<=41; i+=2)
       bar(hp, i);
    return 0;

}
```

Assuming HP is global variable and holding the value NULL . What will be the content of the list

A.34 → 36 → 38 → 40 → NULL

B.40 → 38 → 36 → 34 → NULL

C.41 → 39 → 37 → 35 → NULL

D.34 → 38 → 36 → 40 → NULL

Slide

Consider the function

```
node *hp;
void bar(node *s, int x){
    node *p =getnode(x);

        p->next = s;
        hp = p;
}
int main(){
    for(int i = 34; i<=41; i+=2)
        bar(hp, i);
return 0;

}
```

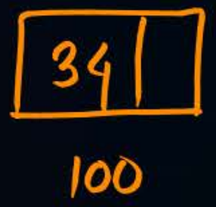
h: 200

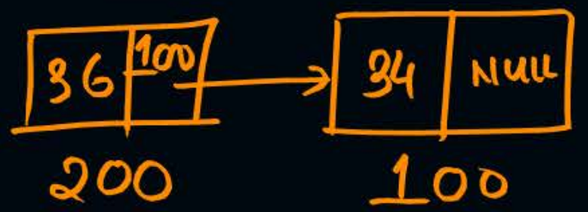
i = 34

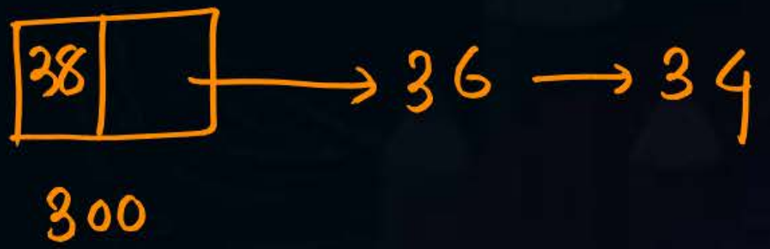
i = 36

i = 38

i = 40







40 → 38 → 36 → 34

Slide

```
Consider the function
node *hp;
void bar(node *s, int x){
    node *p =getnode(x);

        p->next = s;
        hp = p;
}
int main(){
    for(int i = 34; i<=41; i+=2)
        bar(hp, i);
return 0;

}
```

Adding anode at beginning of List

Does not depends upon length of list $O(1) = \Omega(1)$

$\Theta(1)$

Slide

Consider the function (4)

```
void foo(node *s, int x){
    while(s->data!=x
        s =s-> next;
    if(s->data == x)
        printf("x is present");
    else
        printf("x is not preset");
}
```

Searching element in the list if x is 4

1 (100) → 2 (200) → 3 (300) → 4 (400)

(II)

1!=4	2!=4	3!=4	4!=4
S=200	S=300	S=400	false

X is present

Slide

Searching element in the list

X=5

1 → 2 → 3 → 4

100 200 300 400

```
Consider the function
void foo(node *s, int x){
    while(s->data!=x                    )
        s =s-> next;
    if(s->data == x)
        printf("x is present");
    else
        printf("x is not preset");
}
```

1!=5	2!=5	3!=5	4!=5
S=200	S=300	S=400	S=S→next
			S=NULL

NULL→data

NULL pointer dereferencing

Slide

Searching element in the list

```
Consider the function
void foo(node *s, int x){
    while(s->data!=x && s-> next !=NULL)
        s =s-> next;
    if(s->data == x)
        printf("x is present");
    else
        printf("x is not preset");
}
```

X=5

1 → 2 → 3 → 4

100 200 300 400

1!=5	2!=5	3!=5	4!=5 && 4→next != NULL
S=200	S=300	S=400	false

Slide

Consider the following function

```
void foo(node*s, int x, int y){
node*p = getnode(x);
if(s){
        while(s->next!=NULL && s->data!=y)
                s= s->next;
        if(s->data== y){
                p->next = s->next;
                s->next = p;
         }
        else{
        printf("y is not present in the list");
        return;
        }
}
}
```

}Slide

If the list contains 60->42->26->12->NULL

What will be the content of the list if we call

Foo(HP, [redacted] 13, 60)

A. 60 -> 13 -> 42 -> 26 -> 12-> NULL

B. 60 -> 42 -> 26 -> 12 -> NULL

C. 13 -> 60 -> 42 -> 26 -> 12

D. 73 -> 42 -> 26 -> 12

Consider the following function

```
void foo(node*s, int x, int y){
node*p = getnode(x);13  60
if(s){

        while(s->next!=NULL && s->data!=y)
                s= s->next;
        if(s->data== y){
                p->next = s->next;
                s->next = p;
         }
        else{
        printf("y is not present in the list");
        return;
        }
}
```

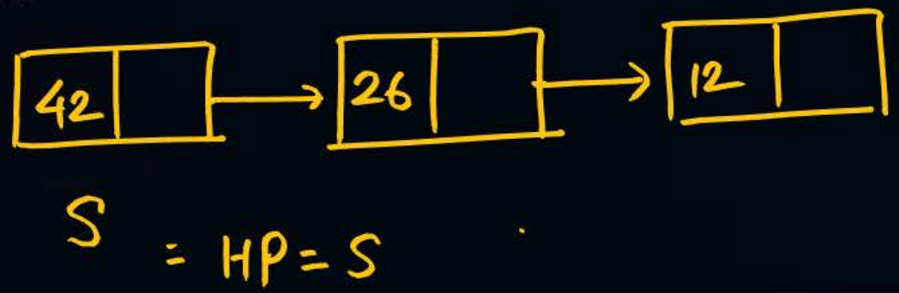
}Slide

Insert x after y

Consider the following program

```
void foo(node *s, int x){
node *p=s,*q;
if(s){
    if(s->data ==x){
     q = s;
     s = s->next;
     free(q);
     HP = s;
}
else{
   while(s->data!=x && s->next !=NULL){
         q = s;
         s = s->next;
   }
   if(s->data == x){
         q -> next = s->next;
         free(s);
}
}
}
```

Non empty



If the list contains 60-> 42->26->12->NULL

What will be the content of the list if we call

Foo(HP, 60)

A. 60 -> 60 -> 42 -> 26 -> 12-> NULL

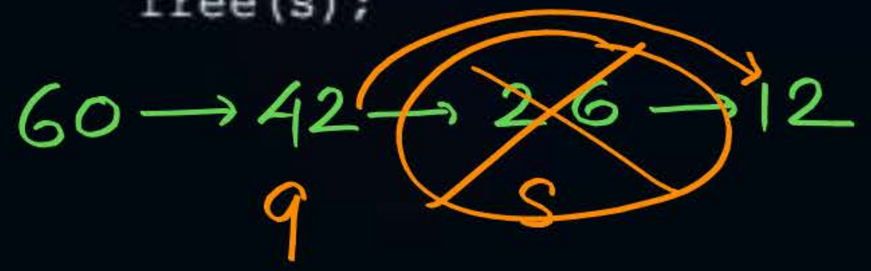
B. 60 -> 42 -> 26 -> 12 -> NULL

C. 42 -> 60 -> 26 -> 12

D. 42 -> 26 -> 12

Consider the following program

```
void foo(node *s, int x){
node *p=s,*q;
if(s){
    if(s->data ==x){
     q = s;
     s = s->next;
     free(q);
     HP = s;
}
else{
   while(s->data!=x && s->next !=NULL){
        q = s;
        s = s->next;
   }
   if(s->data == x){
        q -> next = s->next;
        free(s);
   }
}
```

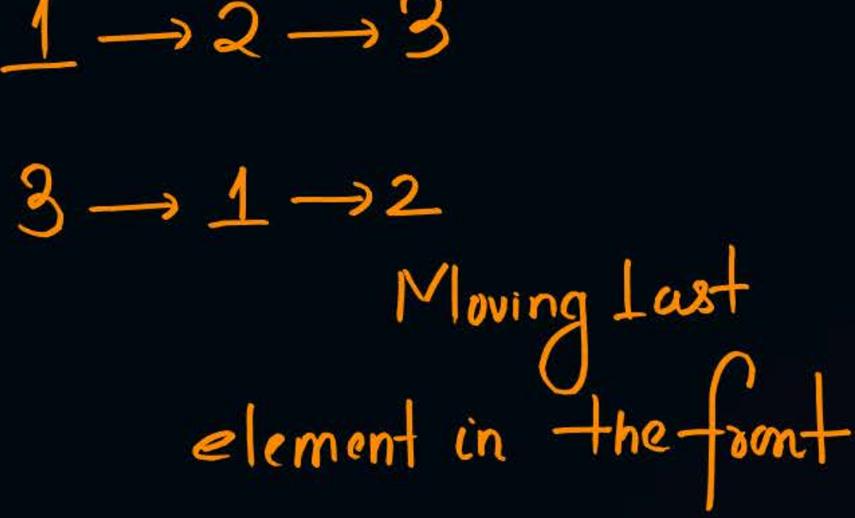


final list 60 → 42 → 12

} 2 pointer traversal
forward pointer and backward pointer

The following C function takes a single-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

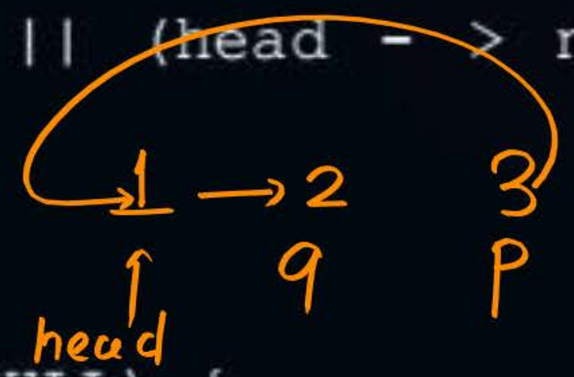
```
typedef struct node {
        int value;
        struct node *next;
}Node;
```



The following C function takes a single-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node {
        int value;
        struct node *next;
}Node;
```

```
Node *move to front (Node *head) {
    Node *p, *q;
    if ((head == NULL) || (head -> next = = NULL))
        return head;
    q= NULL; p=head;
    while (p-> next! = NULL) {
        q = p;
        p = p -> next;
    }
    _____
    return head;
```



2 will be the Last node

q→next = NULL

p→next = head

head = p

Choose the correct alternative to replace the blank line.

(A) `q = NULL; p - > next = head; head = p;`

(B) `q -> next = NULL; head = p; p -> next = head;`

(C) `Head = p; p -> next = q;q -> next = NULL;`

(D) `q -> next = NULL; p -> next = head; head = p;` ✓

Topic : Single Linked List

Consider the C code fragment given below.

```
typedef struct node {
        int data;
        node* next;
}node;
void join (node* m, node* n) {
        node* p = n;
        while (p -> next! = NULL) {
                p = p -> next ;
}
p-> next = m;
}
```

Assuming that m and n terminated linked lists, as m =>3->5->1, n = 6->3->4->9, final n list will be

(A) 3->5->1-> 6->3->4->9

(B) 6->3->4->9-> 3->5->1

C) 3->5->1

(D) 6->3->4->9

(E) None

p

6,3,4,9 → 3→5-1

List m append at the end of list n

Slide

Topic Question

Consider the following function

```
node * bulid123new(){
    node *temp, *temp1=NULL;
    int i =12;
   for (int j=1; j<=4;j++){
        temp = malloc(sizeof(node));
        temp-> data = i++;
        temp->next = temp1;
        temp1= temp;
    }
   return temp;
}
```

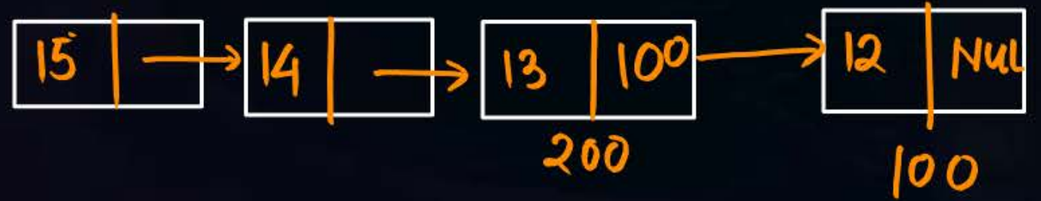
The content of the list is

A. 11->12->13->14->15

B. 11->12->13->14

C. 15->14->13->12->11

D. 15->14->13->12



$temp_1 = 200$

Slide

THANK - YOU